這份教材補的是什麼

上一份《梯度下降法》停在更新式：

$$w^{(k+1)} = w^{(k)} - \eta \frac{dL}{dw}$$

右邊那個 dL/dw 一直是我手推給各位的。參數只有一個的時候推得動，上萬個的時候推不動。

這一份回答的就是**梯度怎麼讓電腦自己算出來**。課本給兩種算法：

	數值微分	倒傳遞演算法
課本章節	4.3 至 4.4 節	第 5 章
寫起來	三行	每種節點各寫一個類別
一次梯度的成本	參數個數 × 2 次正向計算	正向 1 次、反向 1 次
寫錯了會不會知道	幾乎不會寫錯	不會報錯，數字照樣跑得出來
實際用途	拿來驗倒傳遞	真正拿來訓練

兩種都要學，因為**最後要用第一種去驗第二種**。這個動作叫梯度確認，是本份的驗收條件。

倒傳遞演算法 backpropagation（BP）是台灣慣用的名稱，課本簡體版寫作「误差反向传播法」，指的是同一件事。

示範函數和上一份完全一樣：

$$L(w) = (w-3)^2 \qquad \frac{dL}{dw} = 2(w-3)$$

起點 $w = 0$，數字可以和上一份互相核對。中間借課本的「買蘋果」例子學兩條節點規則， 因為那些數字小到可以心算；學會之後再回到 $L(w)$。

怎麼用這份教材：先讀一到四節，做第五、六節的手算練習，自己填完表； 再把程式貼到 Colab 跑，確認跑出來的數字和你填的一樣；最後看第七節。全部做完大約 90 分鐘。

一、方法一：數值微分

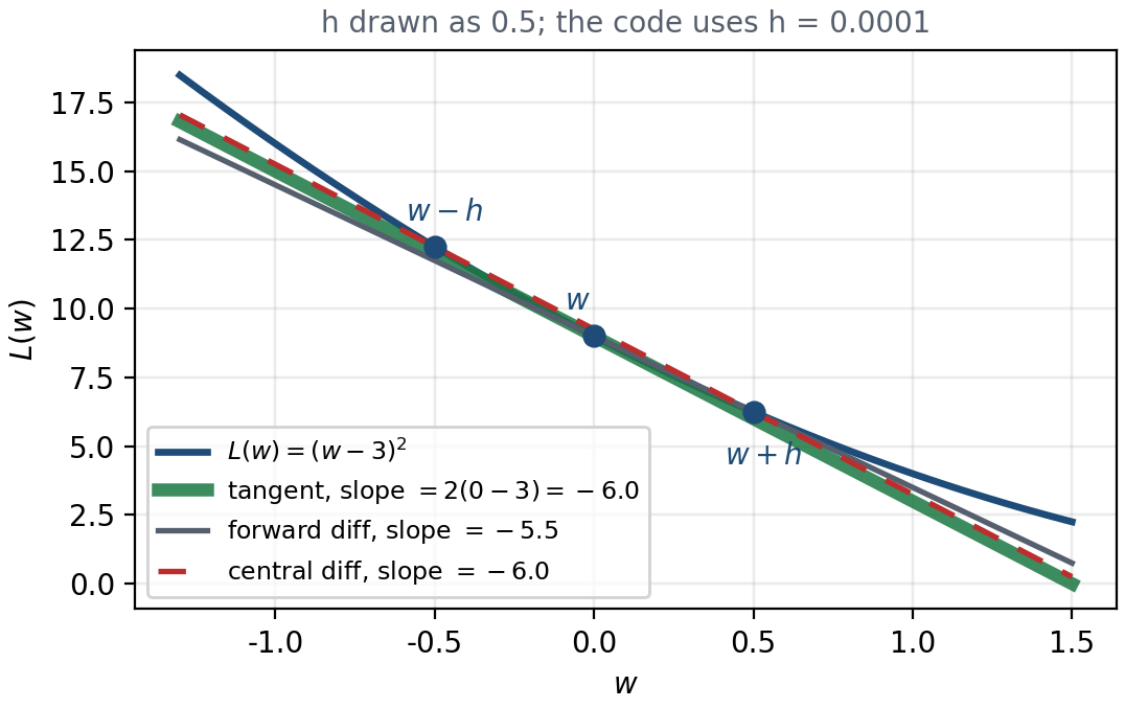
數值微分 numerical differentiation 只用導數的定義，不必推任何公式。 課本式 (4.5) 在 x 左右各取一點，算兩點連線的斜率，叫作中心差分：

$$\frac{df}{dx} \simeq \frac{f(x+h) - f(x-h)}{2h} \qquad h = 10^{-4}$$

在 $w = 0$ 拆開算一次：

步驟	值
$L(0+h)$	8.999400010000
$L(0-h)$	9.000600010000
分子：兩者相減	−0.001200000000
分母：$2h$	0.000200000000
中心差分	−6.000000000013
真正的導數 $2(0-3)$	−6

和真正的 −6 只差浮點數的捨入。



前向差分與中心差分。實際程式用 h = 0.0001，圖上看不出來，所以把 h 放大到 0.5

為什麼取左右兩點，不只取右邊一點？ 只取右邊一點的叫前向差分，在 $w = 0$ 算出來是 −5.9999，差了 10^{-4}。對二次函數，前向差分的誤差剛好等於 h，中心差分沒有這一項。

圖上紅虛線整條疊在綠色切線上，看起來像畫錯，其實是對的：二次函數的中心差分剛好精確。

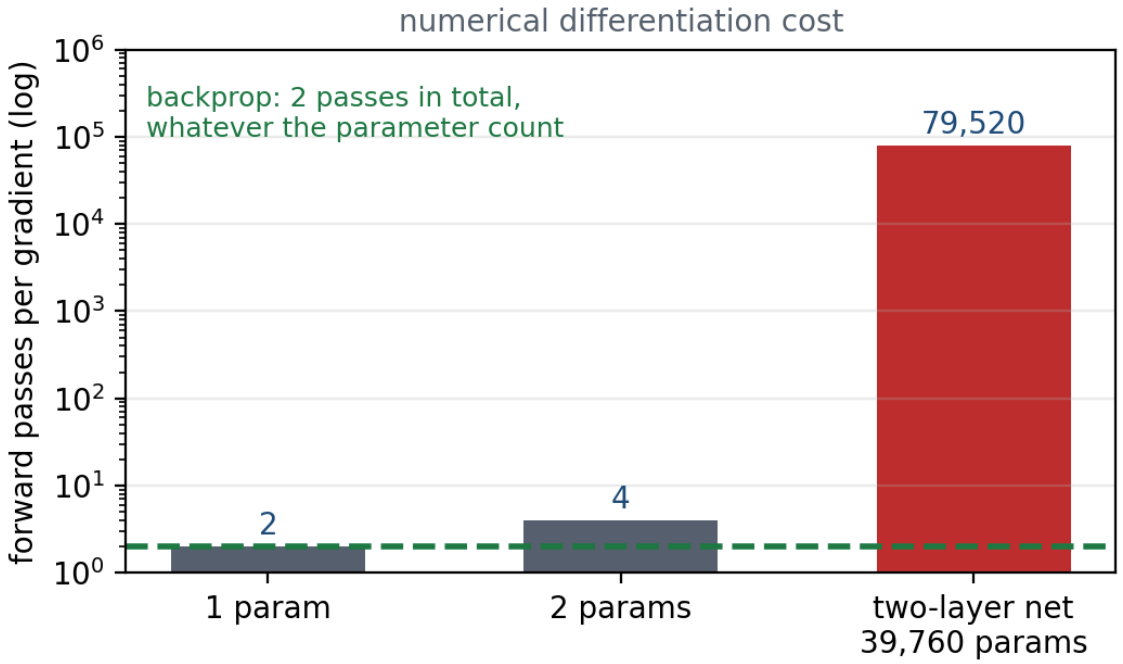
二、數值微分的代價

數值微分準，但是**貴**：每個參數都要把整個網路正向算兩次。

課本 4.5 節的兩層網路是 784–50–10：

	形狀	個數
W1	784 × 50	39,200
b1	50	50
W2	50 × 10	500
b2	10	10
合計		39,760

求一次梯度要正向算 $39{,}760 \times 2 = 79{,}520$ 次。而這只是梯度下降的**一步**。

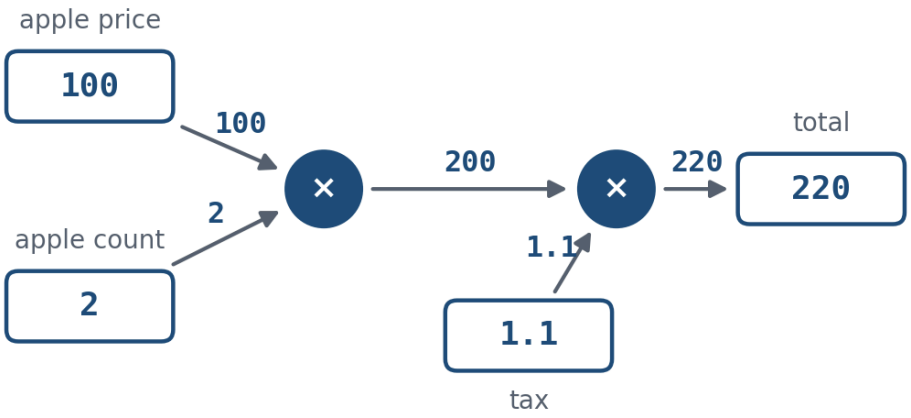


求一次梯度要跑幾次正向計算，對數刻度

倒傳遞不管參數有幾個，都是正向 1 次、反向 1 次。第七節會在同一個網路上實際計時。

三、計算圖：正向傳播

計算圖 computational graph 把一串運算畫成節點與箭頭。課本 5.1 節的例子： 蘋果一個 100 圓，買 2 個，消費稅 1.1。



課本 5.1 節的計算圖，只畫正向。倒傳遞留給第五節自己算

從左往右算叫**正向傳播**：$100 \times 2 = 200$，$200 \times 1.1 = 220$。

局部計算：每個節點只管自己這一格。第一個乘法節點只算 100×2， 它不必知道後面還有消費稅。課本圖 5-4 的例子更清楚：其他東西買了 4000 圓， 加法節點只做 $4000 + 200 = 4200$，前面那 4000 是怎麼來的它完全不管。

這個性質是倒傳遞能成立的原因：整張圖再大，每個節點要處理的事都一樣簡單。

四、鏈式法則與兩條節點規則

現在真正要問的問題是：**蘋果價格動一點，總價會動多少？**也就是 $\partial(\text{總價})/\partial(\text{蘋果價格})$。

倒傳遞從最右邊開始往回走，起點是 1。每經過一個節點， 就把上游傳來的值乘上這個節點的局部導數，這就是鏈式法則 chain rule：

$$\frac{\partial L}{\partial x} = \frac{\partial L}{\partial z} \cdot \frac{\partial z}{\partial x}$$

整張計算圖不管多大，倒傳遞只用得到兩條規則：

節點	正向	局部導數	倒傳遞的規則	課本例子：上游傳來 1.3
加法	$z = x + y$	$\frac{\partial z}{\partial x} = 1,\ \frac{\partial z}{\partial y} = 1$	**原封不動**往兩邊傳	$10 + 5$：兩邊都得到 1.3
乘法	$z = xy$	$\frac{\partial z}{\partial x} = y,\ \frac{\partial z}{\partial y} = x$	乘以**另一邊**的輸入值	10×5：x 那邊得 $1.3 \times 5 = 6.5$，y 那邊得 $1.3 \times 10 = 13$

課本把乘法的規則叫「翻轉值」，程式註解寫「翻轉 x 和 y」，講的是同一件事：往 x 那邊傳的時候乘 y，往 y 那邊傳的時候乘 x。

五、手算練習：蘋果的倒傳遞

用兩條規則把下表填完。起點已經填好。

位置	正向的值	倒傳遞的值	怎麼算出來的
總價	220	1	起點
小計	200		
消費稅	1.1		
蘋果價格	100		
蘋果個數	2		

填完之後對答案：

位置	正向的值	倒傳遞的值	怎麼算出來的
總價	220	1	起點
小計	200	1.1	第二個乘法節點，乘另一邊的消費稅：1×1.1
消費稅	1.1	200	第二個乘法節點，乘另一邊的小計：1×200
蘋果價格	100	2.2	第一個乘法節點，乘另一邊的個數：1.1×2
蘋果個數	2	110	第一個乘法節點，乘另一邊的價格：1.1×100

2.2 是什麼意思

蘋果每漲 1 圓，總價就多 2.2 圓。**自己驗一次**：把蘋果價格改成 101 再正向算，$101 \times 2 \times 1.1 = 222.2$，比 220 多 2.2，和倒傳遞算的一樣。

倒傳遞算出來的導數，就是「輸入動一點，輸出跟著動多少」。

再練一題：蘋果加橘子

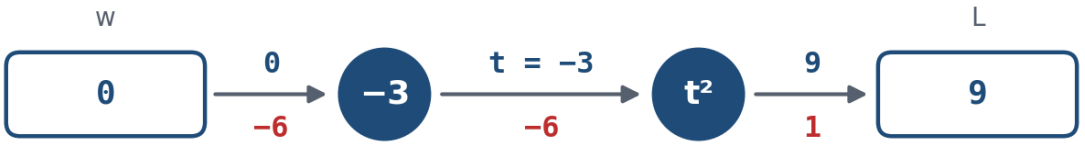
課本圖 5-17：蘋果 2 個、橘子 3 個，橘子每個 150 圓，兩者加總後乘消費稅。 總價是 $(100 \times 2 + 150 \times 3) \times 1.1 = 715$。

求**橘子價格的導數**。中間多了一個加法節點，用第二條規則。

自己驗：橘子漲 1 圓，總價應該從 715 變成 718.3，差就是你算出來的導數。

六、回到 L(w)：用計算圖重算

$L = (w-3)^2$ 拆成兩個節點：先減 3 得到 t，再平方得到 L。



w = 0 的正向（藍）與倒傳遞（紅）。平方節點標成 t²，因為它的輸入是 t

節點	正向	局部導數
減 3	$t = w - 3$	1，和加法節點一樣原封不動往回傳
平方	$L = t^2$	$2t$，所以正向時要記住 t

鏈式法則把兩個局部導數乘起來：

$$\frac{dL}{dw} = \frac{dL}{dt} \cdot \frac{dt}{dw} = 2t \cdot 1 = 2(w-3)$$

和上一份手推的公式一樣。差別在這裡**沒有人推公式**，每個節點只知道自己的局部導數。

把下表填完，第一列已經填好：

w	正向 t	正向 L	倒傳遞 $dL/dt = 2t$	倒傳遞 dL/dw
0	−3	9	−6	−6
0.6				
1.08				

填完之後對答案：

w	正向 t	正向 L	倒傳遞 dL/dt	倒傳遞 dL/dw
0	−3	9	−6	−6
0.6	−2.4	5.76	−4.8	−4.8
1.08	−1.92	3.6864	−3.84	−3.84

最後一欄和《梯度下降法》講義第二節那張表的梯度欄一模一樣。

七、梯度確認

課本 5.7.3 節：倒傳遞很快但容易寫錯，數值微分很慢但不會寫錯，所以拿慢的去驗快的。 這個動作叫梯度確認 gradient check。

判準要先寫下來再跑：兩者差的絕對值小於 10^{-7} 才算通過。

w	數值微分	倒傳遞	差的絕對值	判定
0.000000	−6.000000000013	−6.000000000000	1.27e−11	通過
0.600000	−4.799999999996	−4.800000000000	4.08e−12	通過
1.080000	−3.839999999999	−3.840000000000	6.00e−13	通過
1.464000	−3.072000000000	−3.072000000000	4.80e−13	通過
2.213568	−1.572864000003	−1.572864000000	3.45e−12	通過

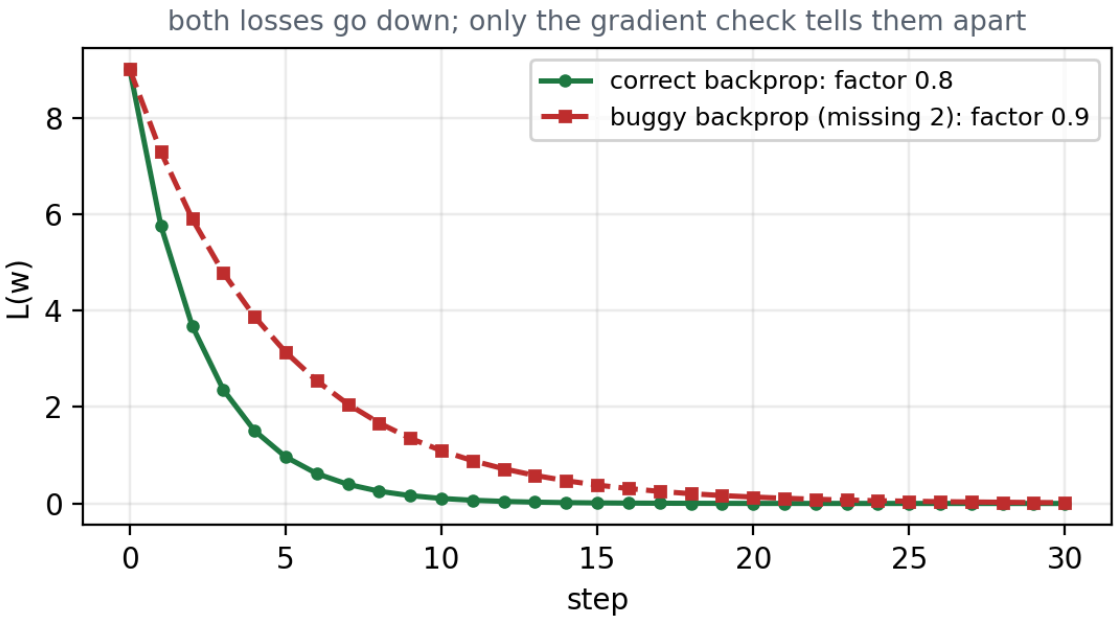
五個 w 是上一份梯度下降實際走過的點。差不會剛好是 0，因為電腦的浮點數精度有限；寫對的話差會非常接近 0，**差很大就代表倒傳遞寫錯了**。

故意寫錯一次

把平方節點的局部導數漏掉 2，寫成 t。這是最常見的錯法之一，程式不會報任何錯。

w	數值微分	寫錯的倒傳遞	差的絕對值	判定
0.000000	−6.000000000013	−3.000000000000	3.00	不通過
0.600000	−4.799999999996	−2.400000000000	2.40	不通過
1.080000	−3.839999999999	−1.920000000000	1.92	不通過

−3 看起來是個很合理的數字。更麻煩的是，**拿它去做梯度下降，損失照樣會降**：



兩條損失曲線都在降，只有梯度確認分得出來

k	正確的 w	寫錯的 w
0	0.000000	0.000000
1	0.600000	0.300000
2	1.080000	0.570000
3	1.464000	0.813000
6	2.213568	1.405677

正確版每一步離 3 的距離剩八成，寫錯版剩九成。寫錯的版本只是收斂得比較慢， **只看損失有沒有下降，你不會發現它錯了**。能抓到它的只有梯度確認。

在兩層網路上做一次

程式第 5 段在課本那個 784–50–10 的網路上，兩種算法各跑一次。課本用 MNIST 的前 3 筆資料， 程式改用固定種子的隨機資料，不必下載資料集，道理完全相同。

	正向次數	時間，作者電腦上量的
數值微分	79,520	約 4 秒
倒傳遞	1，另加反向 1 次	約 0.0002 秒

差兩萬倍上下。時間依機器而定，倍數才是重點，而且這只是梯度下降的一步。

梯度確認依課本的做法，把每組參數兩種算法的差取絕對值再取平均：

參數	差的絕對值平均
W1	7.69e−13
b1	7.91e−13
W2	9.95e−13
b2	1.21e−10

四組落在 10^{-13} 到 10^{-10} 之間，課本用 MNIST 得到的也是這個數量級。

和課本有一處不同：課本的交叉熵在 log 裡加了 10^{-7} 防止 $\log 0$。這一點點會讓損失函數和倒傳遞用的梯度 $(y-t)/N$ 對不上，實測 b2 的差會被拉高到 1.4×10^{-7}，超過判準。程式改用不必加 10^{-7} 的 log-sum-exp 寫法。梯度確認連這麼細的不一致都抓得到，練習 5 讓你自己試。

八、程式：手寫版與框架版

手寫版：每個節點一個類別

課本 5.4 節的乘法層：

```
class MulLayer:
    def forward(self, x, y):
        self.x, self.y = x, y   # 正向時要記住輸入，倒傳遞會用到
        return x * y

    def backward(self, dout):
        dx = dout * self.y      # 往 x 那邊乘 y
        dy = dout * self.x      # 往 y 那邊乘 x
        return dx, dy
```

$L(w)$ 的兩個節點，照同一個樣子寫：

```
class SubConstLayer:            # t = w - 3，局部導數是 1
    def __init__(self, c): self.c = c
    def forward(self, w):  return w - self.c
    def backward(self, dout): return dout * 1

class SquareLayer:              # L = t^2，局部導數是 2t
    def forward(self, t):  self.t = t; return t ** 2
    def backward(self, dout): return dout * 2 * self.t

sub, sq = SubConstLayer(3.0), SquareLayer()
sq.forward(sub.forward(0.0))    # 正向
print(sub.backward(sq.backward(1.0)))   # 倒傳遞從 1 開始，印出 -6.0
```

`backward` 的呼叫順序和 `forward` 相反，這就是「倒」傳遞。

課本程式的註解寫 `220` 與 `2.2 110 200`，實際跑出來是 `220.00000000000003` 與 `110.00000000000001`。這是 1.1 在二進位裡存不精確造成的捨入，不是算錯。

框架版：GradientTape 就是倒傳遞

```
w = tf.Variable(0.0, dtype=tf.float64)
with tf.GradientTape() as tape:
    t = w - 3.0
    L = t ** 2
print(tape.gradient(L, w))      # -6.0
```

`GradientTape` 把正向計算錄下來，再從最後面往回套鏈式法則， 做的就是上面那幾行的事，只是每一種運算的局部導數已經替你寫好了。 用 float64 跑，五個點和手寫版逐位相同，差是 0。

九、AI 時代，人在這裡做什麼

框架內建的運算不會漏掉那個 2，它們的局部導數都驗過了。會出錯的是**框架沒有替你寫的部分**：

情境	為什麼容易錯
自己寫的層	局部導數要自己推，漏一個係數、少轉置一次都不會報錯
自己寫的損失函數	像第七節那個 10^{-7}，損失和梯度差一點點就對不上
自訂梯度	為了數值穩定或加速而手寫的 backward，最容易和 forward 不一致
生成式 AI 寫的 backward	語法正確、形狀正確、數字看起來合理，但係數錯了

這四種錯的共同點是：**程式照跑，損失照降，只是學得比較慢或學到錯的地方**。 第七節那張圖就是證據。

所以判斷「這份 backward 可不可以用」的責任在人身上，而判斷的方法就是梯度確認： 先寫下判準，再拿數值微分去比，差超過判準就不能交出去。這一步花幾秒鐘， 省下的是好幾天的「模型為什麼不收斂」。

十、練習

1. 第五節的蘋果加橘子：求橘子價格的導數，再用「橘子漲 1 圓」自己驗。
2. 把 $L(w) = (w-3)^2$ 改成 $(w-5)^2$。計算圖哪一個節點的局部導數會變？倒傳遞的結果會變嗎？先預測再跑。
3. 寫一個 `CubeLayer`，正向是 t^3。它的 `backward` 要乘什麼？用程式裡的 `gradient_check` 驗你寫的對不對。
4. 程式第 5 段把 h 改成 10^{-2} 再做一次梯度確認，差會變大還是變小？改成 10^{-10} 呢？
5. 把 `SoftmaxWithLoss.forward` 改回課本的 `np.log(self.y + 1e-7)`，再做一次梯度確認。哪一組參數的差變最大？

跑之前先寫下預測，再跑。預測錯比預測對更有價值，那代表你對這個演算法的理解有一個洞。

程式取用

完整程式共 6 段，含梯度確認、故意寫錯、兩層網路計時與 TensorFlow 對照：

- **直接在瀏覽器跑**：https://colab.research.google.com/github/pychang-ai/115-1_DLMA/blob/main/topics/backprop/backprop.ipynb 從課程 repo 的 README 進階主題那一列點「在 Colab 開啟」也是同一個連結。 不必下載、不必安裝、不需要圖形處理器；要改內容請先在 Colab 選「複製到雲端硬碟」， 否則改動不會留下來。
- 本機執行：`python code/backprop.py`，需要 numpy、matplotlib、tensorflow

數值微分那一格要跑幾秒鐘，是正常的，那就是第二節講的代價。

圖上的文字一律用英文。Colab 的預設環境沒有中文字型，用中文會變成方框。